Коротка версія для усного захисту

У програмі реалізовано функцію мого варіанту 16 двома способами. Перша функція func_value1/1 використовує guard-умови, а друга func_value2/1 використовує оператор if. Якщо x > 2, обчислюється факторіал числа. Якщо 1 < x < 2, обчислюється сума k*x для k від 1 до 7. Для факторіала використано хвостову рекурсію, а для суми показано два варіанти: пряму і хвостову рекурсію. Якщо аргумент не належить області визначення, програма повертає повідомлення про помилку.

У даній програмі реалізовано розгалужений алгоритм мовою Erlang для функції мого варіанту 16. За умовою задачі функція має дві гілки. Якщо значення `x` більше 2, потрібно обчислити факторіал `x!`. Якщо значення `x` знаходиться в межах від 1 до 2, тобто `1 < x < 2`, потрібно обчислити суму вигляду `k*x` для `k` від 1 до 7. В усіх інших випадках функція вважається заданою поза областю визначення, тому програма повертає повідомлення про помилку.

На початку програми задається модуль `lab2_var16` і список експортованих функцій. Це означає, що зовні можна викликати функції `func_value1/1`, `func_value2/1` і `factorial/1`. Основна ідея програми полягає в тому, що одна й та сама математична функція реалізується двома способами: через guard-вирази і через оператор `if`.

Функція `func_value1/1` є реалізацією через guard. У першій гілці перевіряється, що `X` є цілим числом і що `X > 2`. Якщо ця умова виконується, викликається функція `factorial(X)`. У другій гілці перевіряється, що `X` є числом і що воно належить інтервалу `1 < X < 2`. У такому випадку викликається функція `sum_direct(X, 7)`, яка обчислює суму через пряму рекурсію. Якщо жодна з умов не виконується, повертається помилка `{error, out_of_domain}`.

Функція `func_value2/1` реалізує ту саму саму задачу, але вже через оператор `if`. Спочатку перевіряється, що аргумент є числом. Далі всередині `if` перевіряються умови. Якщо `X` є цілим числом і `X > 2`, обчислюється факторіал. Якщо `X` належить проміжку `1 < X < 2`, викликається функція `sum_tail(X, 7, 0)`, яка обчислює суму через хвостову рекурсію. Якщо жодна умова не підходить, повертається помилка про вихід за область визначення. Таким чином, `func_value1/1` і `func_value2/1` є еквівалентними за результатом, але відрізняються способом запису умов.

Далі в програмі реалізовано функцію `factorial/1`. Вона перевіряє, що аргумент є цілим невід’ємним числом. Якщо це так, керування передається у допоміжну функцію `factorial_tail/2`, яка обчислює факторіал за допомогою хвостової рекурсії. У цій функції використовується акумулятор `Acc`, у якому накопичується проміжний результат. Коли значення `N` доходить до нуля, накопичений результат повертається як остаточний факторіал. Якщо ж у `factorial/1` передано неправильний аргумент, повертається повідомлення `{error, bad_factorial_arg}`.

Для обчислення суми в програмі використано дві різні реалізації. Перша функція `sum_direct/2` демонструє пряму рекурсію. Вона працює так: на кожному кроці береться поточне значення `I * X`, а потім додається результат рекурсивного виклику для `I - 1`. Базовий випадок настає тоді, коли `I = 0`, і тоді повертається 0. Таким чином формується сума від 1 до 7.

Друга функція `sum_tail/3` демонструє хвостову рекурсію. Тут також обчислюється сума `I * X`, але результат не відкладається до повернення з рекурсії, а одразу додається в акумулятор `Acc`. Потім виконується рекурсивний виклик для `I - 1`. Коли `I` стає рівним нулю, повертається вже накопичений результат. Цей підхід є більш ефективним з точки зору виконання, тому що хвостова рекурсія краще оптимізується.

Отже, у цій програмі показано одразу кілька важливих можливостей мови Erlang. По перше, реалізацію розгалуження через guard-вирази. По друге, реалізацію розгалуження через оператор `if`. По третє, використання рекурсії для обчислень замість циклів. По четверте, показано два види рекурсії для знаходження суми: пряму і хвостову. У результаті програма повністю реалізує умову мого варіанту і правильно працює для всіх перевірених прикладів.